Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

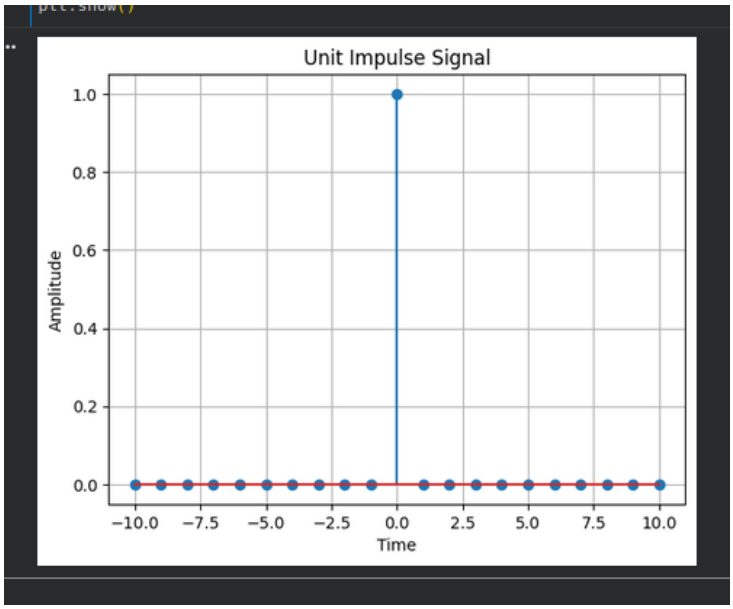
Git hub : https://github.com/mankumarbhalodiya/Digital-Signal-and-Image-Processing/tree/main/Experiment%201

AIM: Simulate Discrete time Sequences

```
import numpy as np
import matplotlib.pyplot as plt
def unit_impulse(length, position):
    signal = np.zeros(length)
    signal[position] = 1
    return signal
# Parameters
start = -10   # Start value of the x-axis range
stop = 10     # Stop value of the x-axis range
step = 1      # Step size
# Generate x-axis values
x = np.arange(start, stop + step, step)
# Generate unit impulse signal
impulse_signal = unit_impulse(len(x), abs(start) // step)
# Plot the signal
plt.stem(x, impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.grid(True)
plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

Output :



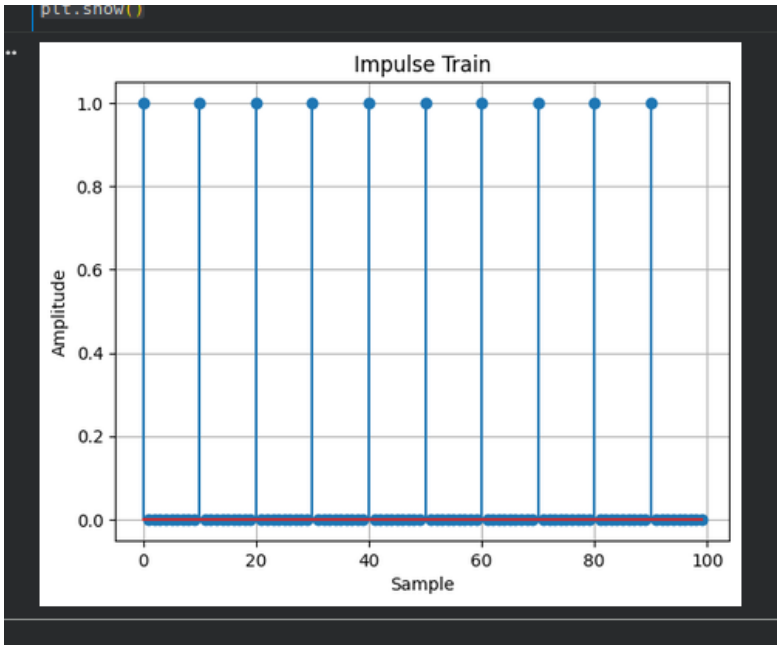
AIM : Simulate impulse train

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_impulse_train(signal_length, period):
    impulse_train = np.zeros(signal_length)
    for n in range(signal_length):
        if n % period == 0:
            impulse_train[n] = 1
    return impulse_train

# Define the parameters for the impulse train
signal_length = 100   # Length of the impulse train
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
period = 10        # Period of the impulse train

# Simulate the impulse train

impulse_train = simulate_impulse_train(signal_length, period)

# Plot and display the impulse train

plt.stem(impulse_train)

plt.title('Impulse Train')

plt.xlabel('Sample')

plt.ylabel('Amplitude')

plt.grid(True)

plt.show()
```

Output :



AIM: Write a python program to Simulate continuous and discrete unit step signal,

```
import numpy as np
```

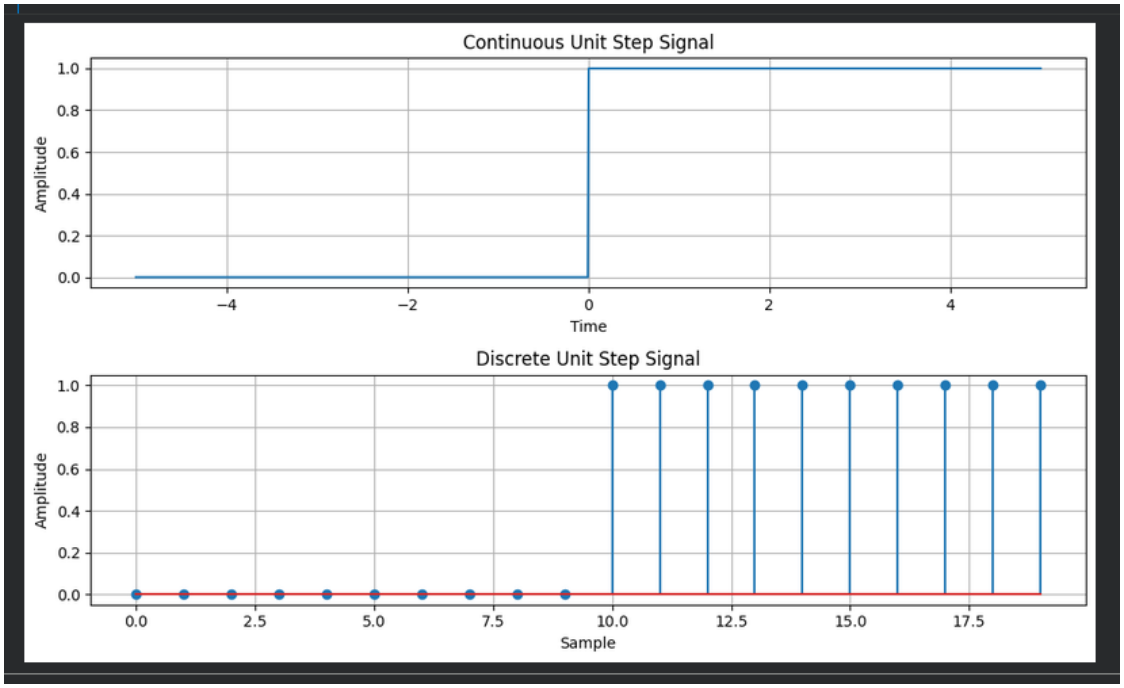
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
import matplotlib.pyplot as plt
def simulate_continuous_unit_step(time):
    unit_step = np.zeros_like(time)
    unit_step[time >= 0] = 1
    return unit_step
def simulate_discrete_unit_step(num_samples):
    unit_step = np.zeros(num_samples)
    unit_step[num_samples // 2:] = 1
    return unit_step
time = np.linspace(-5, 5, 1000)
continuous_unit_step = simulate_continuous_unit_step(time)
num_samples = 20
discrete_unit_step = simulate_discrete_unit_step(num_samples)
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step)
plt.title('Continuous Unit Step Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid(True)
plt.subplot(2, 1, 2)
plt.stem(discrete_unit_step)
plt.title('Discrete Unit Step Signal')
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output :



AIM: Write a python program to Simulate continuous and discrete ramp signal,

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_ramp(time, slope):
    ramp = np.zeros_like(time)
    ramp[time >= 0] = slope * time[time >= 0]
    return ramp
```

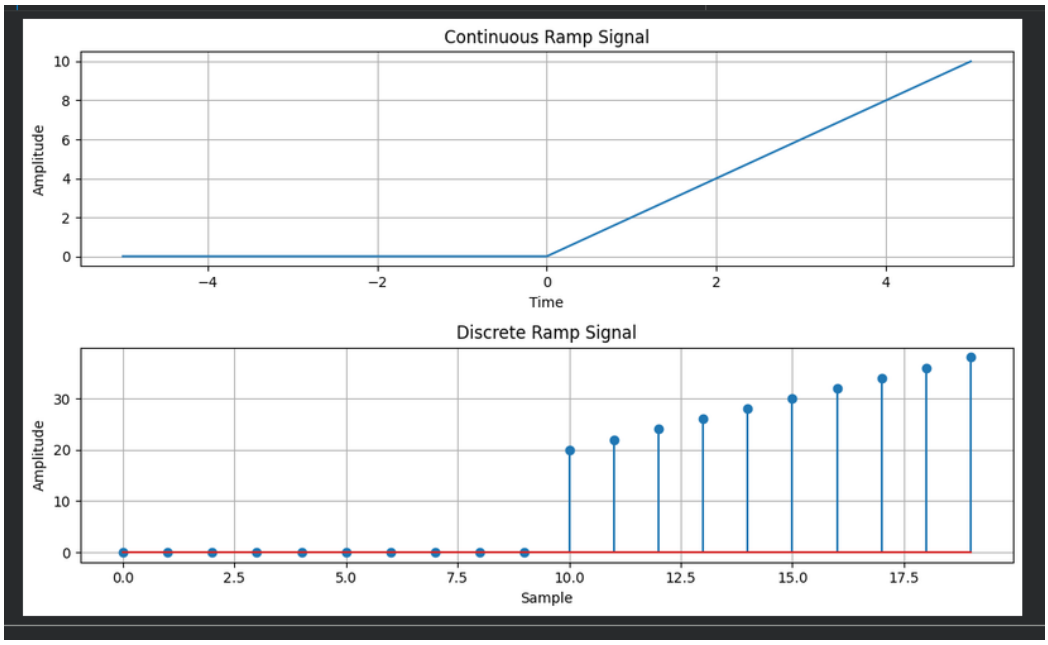
Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
def simulate_discrete_ramp(num_samples, slope):
    ramp = np.zeros(num_samples)
    ramp[num_samples // 2:] = slope * np.arange(num_samples // 2, num_samples)
    return ramp
time = np.linspace(-5, 5, 1000)
num_samples = 20
slope = 2
continuous_ramp = simulate_continuous_ramp(time, slope)
discrete_ramp = simulate_discrete_ramp(num_samples, slope)
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid(True)
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.grid(True)
plt.tight_layout()
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

plt.show()

Output :



AIM: Write a python program to Simulate continuous and discrete exponential signal,

import numpy as np import matplotlib.pyplot as plt

def simulate_continuous_exponential(time, amplitude, coefficient): exponential_signal = amplitude * np.exp(coefficient * time) return exponential_signal def simulate_discrete_exponential(num_samples, amplitude, coefficient): exponential_signal = amplitude * np.exp(coefficient * np.arange(num_samples)) return exponential_signal

time = np.linspace(0, 5, 1000) num_samples = 20 amplitude = 2 coefficient = -0.5

continuous_exponential = simulate_continuous_exponential(time, amplitude, coefficient) discrete_exponential = simulate_discrete_exponential(num_samples, amplitude, coefficient)

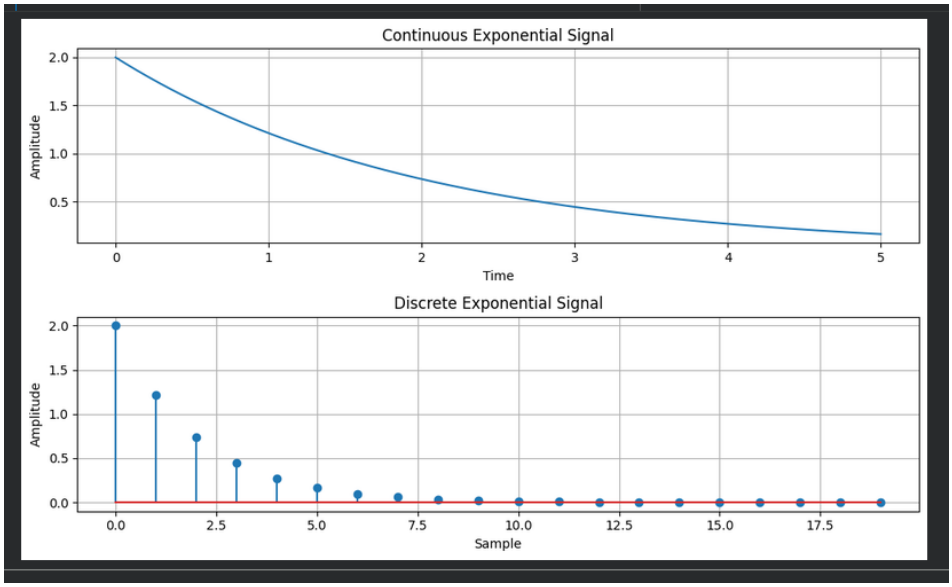
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
plt.plot(time, continuous_exponential)

plt.title('Continuous Exponential Signal')

plt.xlabel('Time')

plt.ylabel('Amplitude')

plt.grid(True)

plt.subplot(2, 1, 2)

plt.stem(discrete_exponential)

plt.title('Discrete Exponential Signal') plt.xlabel('Sample')

plt.ylabel('Amplitude')

plt.grid(True)

plt.tight_layout() plt.show()
```

Output :



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

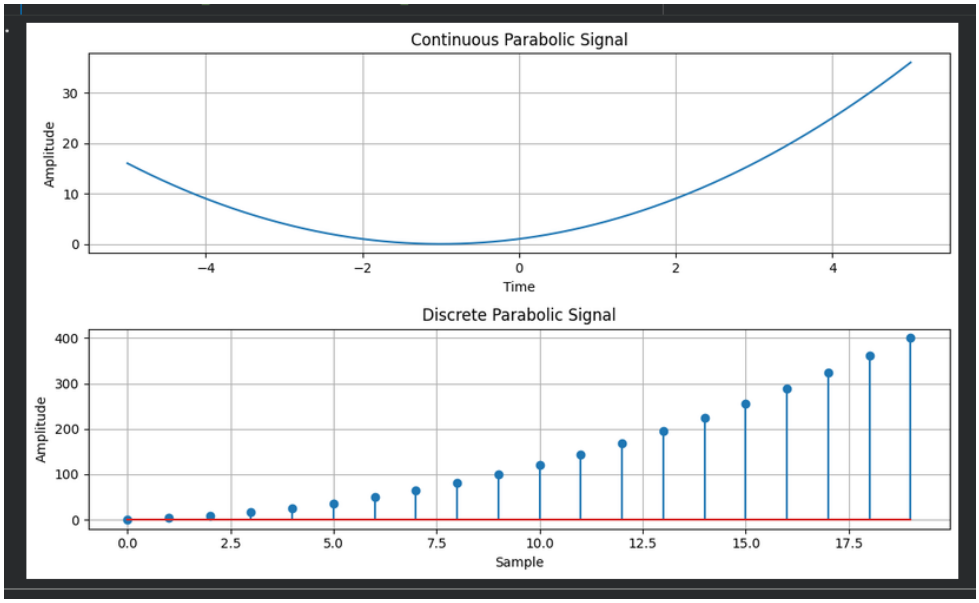
AIM: Write a python program to Simulate continuous and discrete parabolic signal

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_parabolic(time, coefficients):
    parabolic_signal = np.polyval(coefficients, time)
    return parabolic_signal
def simulate_discrete_parabolic(num_samples, coefficients):
    parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
    return parabolic_signal
time = np.linspace(-5, 5, 1000)
num_samples = 20
coefficients = [1, 2, 1]
continuous_parabolic = simulate_continuous_parabolic(time, coefficients)
discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid(True)
plt.subplot(2, 1, 2)
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
plt.stem(discrete_parabolic)

plt.title('Discrete Parabolic Signal')

plt.xlabel('Sample')

plt.ylabel('Amplitude')

plt.grid(True)

plt.tight_layout()

plt.show()

# np.savetxt('continuous_parabolic.txt', continuous_parabolic, delimiter=',')

# np.savetxt('discrete_parabolic.txt', discrete_parabolic, delimiter=',')
```

Output :



AIM: Write a python program to Simulate continuous and discrete sine wave signal

```
import numpy as np import matplotlib.pyplot as plt

def simulate_continuous_sine_wave(time, amplitude, frequency, phase): sine_wave =
amplitude * np.sin(2 * np.pi * frequency * time + phase) return sine_wave
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

def simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude, frequency, phase): time = np.arange(num_samples) / sampling_frequency sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase) return sine_wave

time = np.linspace(0, 1, 1000) num_samples = 100 sampling_frequency = 10 amplitude = 1 frequency = 2 phase = 0

continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude, frequency, phase) discrete_sine_wave = simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude, frequency, phase)

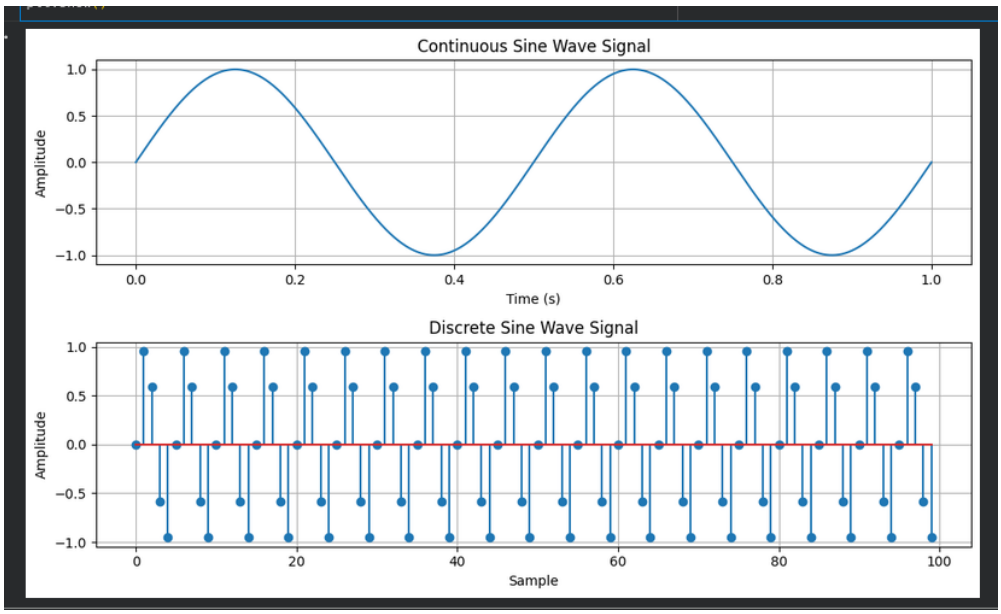
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1) plt.plot(time, continuous_sine_wave) plt.title('Continuous Sine Wave Signal') plt.xlabel('Time (s)') plt.ylabel('Amplitude') plt.grid(True)

plt.subplot(2, 1, 2) plt.stem(discrete_sine_wave) plt.title('Discrete Sine Wave Signal') plt.xlabel('Sample') plt.ylabel('Amplitude') plt.grid(True)

plt.tight_layout() plt.show()

Output :



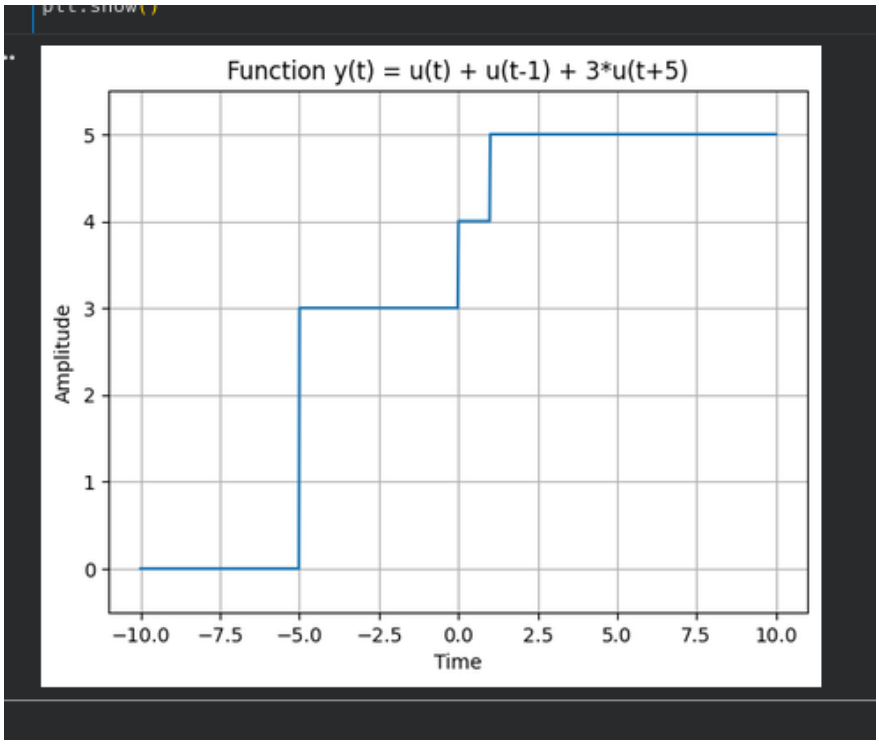
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

AIM: Write a python program to simulate y(t)=u(t)+u(t-1)+3u(t+5).

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time >= 0] = 1
    y[time >= 1] += 1
    y[time >= -5] += 3
    return y
time = np.linspace(-10, 10, 1000)
function_values = simulate_function(time)
plt.plot(time, function_values)
plt.title('Function y(t) = u(t) + u(t-1) + 3*u(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```

Output :

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**



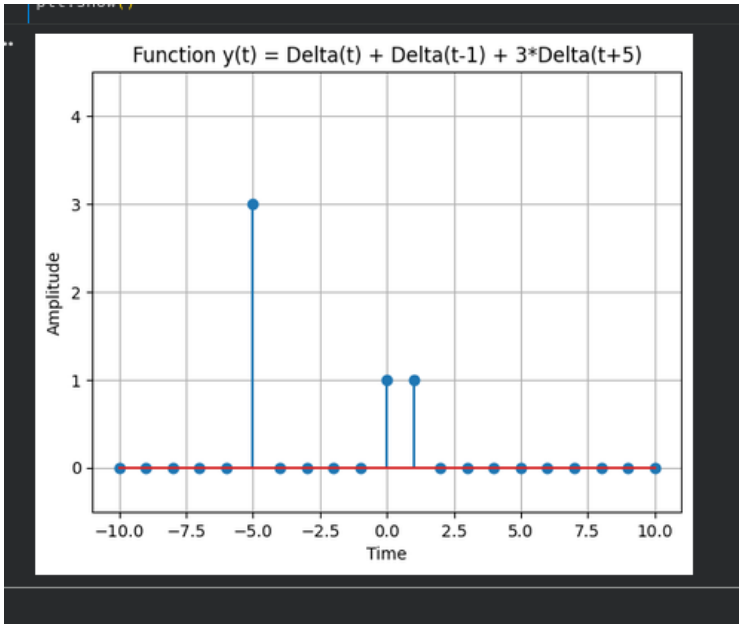
AIM: Write a python program to simulate y(t)=Delta(t)+delta(t-1)+3*delta(t+5).

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time == 0] = 1
    y[time == 1] += 1
    y[time == -5] += 3
    return y
time = np.arange(-10, 11)
function_values = simulate_function(time)
plt.stem(time, function_values)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Simulate discrete time sequences.**	
Experiment No: 1	**Date: 14/07/2026**	**Enrolment No: 92400133125**

```
plt.title('Function y(t) = Delta(t) + Delta(t-1) + 3*Delta(t+5)')

plt.xlabel('Time')

plt.ylabel('Amplitude')

plt.ylim([-0.5, 4.5])

plt.grid(True)

plt.show()
```

Output :



Conclusion:-

We started this experiment to learn how to simulate and visualize discrete and continuous-time signals using Python,

NumPy, and Matplotlib. Through this, we understood basic signal types like impulse, step, ramp, and sine waves,

which are essential for grasping core concepts in digital signal processing.